

Sprint 2 — Autenticación y multi-usuario

API REST con autenticación JWT, soporte multi-usuario y arquitectura con use cases

Stack: Node.js · Express · Prisma · MySQL (Aiven) · Zod · bcrypt · JWT · ES Modules

Índice

- 1. [Resumen del Sprint 2](#)
- 2. [Nuevas dependencias](#)
- 3. [Modelo de datos ampliado](#)
- 4. [Arquitectura con use cases](#)
- 5. [Nuevos endpoints](#)
- 6. [Middlewares](#)
- 7. [Módulo auth completo](#)
- 8. [Módulo games actualizado](#)
- 9. [Clases de error](#)
- 10. [Manejo de errores](#)
- 11. [Pruebas con Bruno](#)
- 12. [Verificación en MySQL Workbench](#)
- 13. [Capturas recomendadas](#)

1. Resumen del Sprint 2

En este sprint llevamos la API al siguiente nivel añadiendo:

- **Registro de usuarios** con email, contraseña (hasheada con bcrypt) y alias
- **Login** que devuelve un token JWT
- **Protección de rutas** — solo usuarios autenticados acceden a `/api/games`
- **Aislamiento de datos** — cada usuario ve y gestiona solo sus juegos
- **Arquitectura con use cases** — separación limpia entre controller, lógica de negocio y acceso a datos
- **Clases de error tipadas** — `NotFoundError`, `ConflictError`, `UnauthorizedError`

2. Nuevas dependencias

Se añadieron dos paquetes para autenticación:

```
npm install bcrypt jsonwebtoken
```

Paquete	Versión	Uso
bcrypt	5.x	Hash de contraseñas (12 salt rounds)
jsonwebtoken	9.x	Generación y verificación de tokens JWT

package.json final

```
{
  "dependencies": {
    "@prisma/client": "^6.1.0",
    "bcrypt": "^5.x",
    "cors": "^2.8.3",
    "dotenv": "^16.4.1",
    "express": "^4.21.1",
    "helmet": "^8.1.1",
    "jsonwebtoken": "^9.x",
    "morgan": "^1.10.1",
    "prisma": "^6.1.0",
    "zod": "^3.25.0"
  }
}
```

3. Modelo de datos ampliado

3.1 Nuevo modelo User

```
model User {
  id          Int          @id @default(autoincrement())
  email       String       @unique
  password    String
  alias       String
  createdAt   DateTime     @default(now())
  updatedAt   DateTime     @updatedAt
  games       Game[]
}
```

3.2 Game actualizado con userId

```
model Game {
  id          Int          @id @default(autoincrement())
  name        String
  category    String
  tags        String       @db.Text
  metacriticScore Int
  hoursToBeat Float
  completed   Boolean      @default(false)
  completedAt DateTime?
  notes       String?      @db.Text
  rating      Int?
  priorityScore Float       @default(0)
  createdAt   DateTime     @default(now())
  updatedAt   DateTime     @updatedAt

  userId Int
}
```

```
    user    User @relation(fields: [userId], references: [id])
  }
}
```

Cambio clave: `userId` es obligatorio. Cada juego pertenece a un usuario. Se eliminaron los datos de prueba del Sprint 1 antes de migrar.

3.3 Migración

```
# 1. Limpiar datos de prueba del Sprint 1
npx prisma db execute --schema=prisma/schema.prisma --stdin <<< "DELETE FROM Game;"

# 2. Ejecutar migración con el nuevo modelo
npx prisma migrate dev --name add-users
```

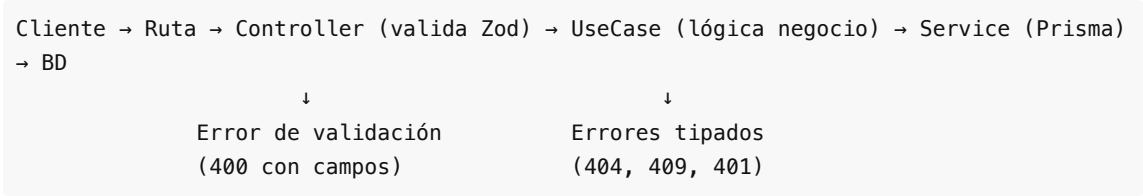
4. Arquitectura con use cases

4.1 Estructura completa del proyecto

```
src/
├─ config/
│  └─ env.js           Variables de entorno
│  └─ prisma.js        Cliente Prisma singleton
├─ docs/               Documentación técnica
├─ errors/
│  └─ index.js         Clases AppError, NotFoundError, ConflictError,
UnauthorizedError
├─ middlewares/
│  └─ auth.middleware.js Protege rutas con JWT
│  └─ error.middleware.js Captura ZodError + AppError + genéricos
│  └─ not-found.middleware.js 404
├─ modules/
│  └─ auth/
│     └─ auth.routes.js   POST /register, POST /login
│     └─ auth.controller.js Valida Zod, llama al use case, responde
│     └─ auth.useCases.js  Lógica de negocio (register, login)
│     └─ auth.service.js   Solo consultas Prisma
│     └─ auth.schemas.js  Schemas Zod
│  └─ games/
│     └─ games.routes.js   Rutas REST protegidas
│     └─ games.controller.js Valida Zod, llama al use case, responde
│     └─ games.useCases.js  Lógica de negocio + ownership + prioridad
│     └─ games.service.js   Solo consultas Prisma
│     └─ games.schemas.js  Schemas Zod
├─ routes/
│  └─ index.js           Router principal
├─ utils/
│  └─ jwt.js             signToken / verifyToken
│  └─ priority.js        Algoritmo de prioridad
```

— app.js	Configuración Express
— server.js	Punto de entrada

4.2 Flujo de cada request



4.3 Responsabilidad de cada capa

Capa	Responsabilidad
routes.js	Define endpoints y aplica middlewares
controller.js	Valida input con Zod, llama al use case, formatea respuesta
useCases.js	Lógica de negocio, reglas, orquestación, lanza errores tipados
service.js	Solo consultas a base de datos con Prisma (find, create, update, delete)
schemas.js	Schemas de validación Zod

5. Nuevos endpoints

5.1 POST /api/auth/register — Crear cuenta

Registra un nuevo usuario. La contraseña se hashea con bcrypt (12 rounds). Se devuelve un token JWT.

Body:

```
{
  "email": "gamer@example.com",
  "password": "miPassword123",
  "alias": "SuperGamer"
}
```

Respuesta (201):

```
{
  "success": true,
  "data": {
    "token": "eyJhbGciOiJIUzI1NiIs...\"",
    "user": {
      "id": 1,
      "email": "gamer@example.com",
      "alias": "SuperGamer"
    }
  }
}
```

```
}  
}
```

Validación Zod:

```
export const registerSchema = z.object({  
  email: z.string().email('Email inválido'),  
  password: z.string().min(8, 'La contraseña debe tener al menos 8 caracteres'),  
  alias: z.string().min(2, 'El alias debe tener al menos 2 caracteres').max(30),  
});
```

Errores:

- 409 si el email ya está registrado
- 400 si los datos no pasan la validación Zod

5.2 POST /api/auth/login — Iniciar sesión

Verifica credenciales y devuelve un token JWT.

Body:

```
{  
  "email": "gamer@example.com",  
  "password": "miPassword123"  
}
```

Respuesta (200):

```
{  
  "success": true,  
  "data": {  
    "token": "eyJhbGciOiJIUzI1NiIs...",  
    "user": {  
      "id": 1,  
      "email": "gamer@example.com",  
      "alias": "SuperGamer"  
    }  
  }  
}
```

Errores:

- 401 si email o contraseña incorrectos

5.3 Endpoints protegidos (requieren token)

Todos los endpoints de /api/games ahora requieren autenticación:

Método	Ruta	Auth	Descripción Sprint 2
--------	------	------	----------------------

GET	/api/games	✓ Bearer token	Lista SOLO juegos del usuario autenticado
GET	/api/games/:id	✓ Bearer token	Obtiene juego si pertenece al usuario
POST	/api/games	✓ Bearer token	Crea juego asociado al usuario
PUT	/api/games/:id	✓ Bearer token	Actualiza juego si pertenece al usuario
DELETE	/api/games/:id	✓ Bearer token	Elimina juego si pertenece al usuario
PATCH	/api/games/:id/complete	✓ Bearer token	Marca completado si pertenece al usuario

Header requerido:

```
Authorization: Bearer eyJhbGciOiJIUzI1NiIs...
```

Error si no hay token:

```
{
  "success": false,
  "message": "Token de acceso requerido"
}
```

6. Middlewares

6.1 Auth middleware — src/middlewares/auth.middleware.js

```
import { verifyToken } from '../utils/jwt.js';

export const authMiddleware = (req, res, next) => {
  const authHeader = req.headers.authorization;

  if (!authHeader || !authHeader.startsWith('Bearer ')) {
    return res.status(401).json({
      success: false,
      message: 'Token de acceso requerido',
    });
  }

  try {
    const token = authHeader.split(' ')[1];
    const decoded = verifyToken(token);
    req.user = decoded;
    next();
  } catch (error) {
    return res.status(401).json({
      success: false,
      message: 'Token inválido o expirado',
    });
  }
}
```

```
}  
};
```

Se aplica a todas las rutas de games en `games.routes.js` :

```
gamesRouter.use(authMiddleware);
```

6.2 Error middleware actualizado — `src/middlewares/error.middleware.js`

Ahora captura tres tipos de error:

```
import { ZodError } from 'zod';  
import { AppError } from '../errors/index.js';  
  
export const errorMiddleware = (err, _req, res, _next) => {  
  // 1. Error de validación Zod → 400 con errores por campo  
  if (err instanceof ZodError) {  
    const errors = err.errors.map((e) => ({  
      field: e.path.join('.'),  
      message: e.message,  
    }));  
  
    return res.status(400).json({  
      success: false,  
      message: 'Error de validación',  
      errors,  
    });  
  }  
  
  // 2. Error de aplicación (AppError) → status code personalizado  
  if (err instanceof AppError) {  
    return res.status(err.status).json({  
      success: false,  
      message: err.message,  
    });  
  }  
  
  // 3. Error no controlado → 500  
  console.error(err);  
  res.status(500).json({  
    success: false,  
    message: 'Internal server error',  
  });  
};
```

7. Módulo auth completo

7.1 Auth Service — solo Prisma

```
import { prisma } from '../../config/prisma.js';

export class AuthService {
  async findByEmail(email) {
    return prisma.user.findUnique({ where: { email } });
  }

  async createUser(data) {
    return prisma.user.create({ data });
  }
}
```

7.2 Auth Use Cases — lógica de negocio

```
import bcrypt from 'bcrypt';
import { AuthService } from './auth.service.js';
import { signToken } from '../../utils/jwt.js';
import { ConflictError, UnauthorizedError } from '../../errors/index.js';

const authService = new AuthService();

export class AuthUseCases {
  async register(input) {
    // 1. Validar negocio: email no duplicado
    const existing = await authService.findByEmail(input.email);
    if (existing) {
      throw new ConflictError('El email ya está registrado');
    }

    // 2. Hash de contraseña
    const hashedPassword = await bcrypt.hash(input.password, 12);

    // 3. Crear usuario
    const user = await authService.createUser({
      email: input.email,
      password: hashedPassword,
      alias: input.alias,
    });

    // 4. Generar token JWT
    const token = signToken({ id: user.id, email: user.email, alias: user.alias });

    return { token, user: { id: user.id, email: user.email, alias: user.alias } };
  }

  async login(input) {
    // 1. Buscar usuario
    const user = await authService.findByEmail(input.email);
    if (!user) {
      throw new UnauthorizedError('Credenciales incorrectas');
    }
  }
}
```

```

    }

    // 2. Verificar contraseña
    const validPassword = await bcrypt.compare(input.password, user.password);
    if (!validPassword) {
        throw new UnauthorizedError('Credenciales incorrectas');
    }

    // 3. Generar token JWT
    const token = signToken({ id: user.id, email: user.email, alias: user.alias });

    return { token, user: { id: user.id, email: user.email, alias: user.alias } };
}
}

```

7.3 Auth Controller

```

import { AuthUseCases } from './auth.useCases.js';
import { registerSchema, loginSchema } from './auth.schemas.js';

const authUseCases = new AuthUseCases();

export class AuthController {
    async register(req, res, next) {
        try {
            const parsed = registerSchema.parse(req.body);
            const result = await authUseCases.register(parsed);
            res.status(201).json({ success: true, data: result });
        } catch (error) {
            next(error);
        }
    }

    async login(req, res, next) {
        try {
            const parsed = loginSchema.parse(req.body);
            const result = await authUseCases.login(parsed);
            res.json({ success: true, data: result });
        } catch (error) {
            next(error);
        }
    }
}

```

8. Módulo games actualizado

8.1 Games Service — solo Prisma

```
import { prisma } from '../../config/prisma.js';

export class GamesService {
  async findMany(where, orderBy) {
    return prisma.game.findMany({ where, orderBy });
  }

  async findFirst(where) {
    return prisma.game.findFirst({ where });
  }

  async create(data) {
    return prisma.game.create({ data });
  }

  async update(id, data) {
    return prisma.game.update({ where: { id }, data });
  }

  async delete(id) {
    return prisma.game.delete({ where: { id } });
  }
}
```

8.2 Games Use Cases — lógica con ownership

```
import { GamesService } from './games.service.js';
import { calculatePriorityScore } from '../../utils/priority.js';
import { NotFoundError } from '../../errors/index.js';

const gamesService = new GamesService();

export class GamesUseCases {
  async listGames(filters, userId) {
    const where = { userId };
    // ...filtros dinámicos (search, category, tag, completed)...
    const orderBy = {};
    orderBy[filters.sortBy || 'priorityScore'] = filters.order || 'desc';

    const games = await gamesService.findMany(where, orderBy);
    return games.map(game => ({
      ...game,
      completedAt: game.completedAt?.toISOString() ?? null,
    }));
  }

  async getGameById(id, userId) {
    const game = await gamesService.findFirst({ id, userId });
    if (!game) throw new NotFoundError('Juego no encontrado');
    return { ...game, completedAt: game.completedAt?.toISOString() ?? null };
  }
}
```

```

}

async createGame(input, userId) {
  const priorityScore = calculatePriorityScore(input.metacriticScore,
input.hoursToBeat);
  const game = await gamesService.create({ ...input, userId, priorityScore });
  return { ...game, completedAt: game.completedAt?.toISOString() ?? null };
}

async updateGame(id, input, userId) {
  const existing = await gamesService.findFirst({ id, userId });
  if (!existing) throw new NotFoundError('Juego no encontrado');

  const metacriticScore = input.metacriticScore ?? existing.metacriticScore;
  const hoursToBeat = input.hoursToBeat ?? existing.hoursToBeat;
  const priorityScore = calculatePriorityScore(metacriticScore, hoursToBeat);

  const game = await gamesService.update(id, { ...input, priorityScore });
  return { ...game, completedAt: game.completedAt?.toISOString() ?? null };
}

async deleteGame(id, userId) {
  const existing = await gamesService.findFirst({ id, userId });
  if (!existing) throw new NotFoundError('Juego no encontrado');
  await gamesService.delete(id);
  return { message: 'Juego eliminado correctamente' };
}

async completeGame(id, data, userId) {
  const existing = await gamesService.findFirst({ id, userId });
  if (!existing) throw new NotFoundError('Juego no encontrado');

  const game = await gamesService.update(id, {
    completed: true,
    completedAt: new Date(),
    ...(data.notes !== undefined && { notes: data.notes }),
    ...(data.rating !== undefined && { rating: data.rating }),
  });

  return { ...game, completedAt: game.completedAt?.toISOString() ?? null };
}
}

```

Cambio clave: Cada método recibe `userId` y filtra por él usando `findFirst` en lugar de `findUnique`, asegurando que un usuario solo accede a sus propios juegos.

9. Clases de error

Creadas en `src/errors/index.js` :

```

export class AppError extends Error {
  constructor(message, status) {
    super(message);
    this.status = status;
  }
}

export class NotFoundError extends AppError {
  constructor(message = 'Recurso no encontrado') {
    super(message, 404);
  }
}

export class ConflictError extends AppError {
  constructor(message = 'Conflicto') {
    super(message, 409);
  }
}

export class UnauthorizedError extends AppError {
  constructor(message = 'No autorizado') {
    super(message, 401);
  }
}

```

Uso en use cases:

- `NotFoundError` → juego no encontrado o que no pertenece al usuario
- `ConflictError` → email duplicado en registro
- `UnauthorizedError` → credenciales incorrectas en login

10. Manejo de errores

Error	Código HTTP	Formato	Origen
Validación Zod	400	{ errors: [{ field, message }] }	Controller (Zod)
Email duplicado	409	{ message }	Auth UseCase
Credenciales incorrectas	401	{ message }	Auth UseCase
Token requerido	401	{ message }	Auth Middleware
Token inválido	401	{ message }	Auth Middleware
Juego no encontrado	404	{ message }	Games UseCase
Error interno	500	{ message }	Error middleware

Ruta no encontrada	404	{ message }	Not found middleware
--------------------	-----	-------------	-------------------------

11. Pruebas con Bruno

11.1 Colección actualizada

La colección Bruno en `docs/bruno/` se actualizó con:

Colección: Gamer Backlog API

```

├─ Health
│   └─ Health check          GET  /api/health
├─ Auth
│   ├── Register            POST /api/auth/register
│   └─ Login                POST /api/auth/login
└─ Games
    ├── Crear juego          POST /api/games (Bearer token)
    ├── Listar juegos        GET  /api/games (Bearer token)
    ├── Listar con filtros   GET  /api/games?category=... (Bearer token)
    ├── Obtener juego por ID GET  /api/games/1 (Bearer token)
    ├── Editar juego         PUT  /api/games/1 (Bearer token)
    ├── Eliminar juego       DELETE /api/games/1 (Bearer token)
    ├── Marcar como completado PATCH /api/games/1/complete (Bearer token)
    └─ Probar validacion     POST /api/games (Bearer token, body vacío)

```

11.2 Flujo de prueba

1. Health check → verificar servidor funcionando
2. Register → crear usuario → copiar token
3. Login → verificar que funciona con mismas credenciales
4. Crear juego → con token en header Authorization
5. Listar juegos → ver juegos del usuario
6. Probar filtros → `?category=RPG, ?search=zelda`, etc.
7. Editar juego → PUT con datos parciales
8. Completar juego → PATCH con notas y rating
9. Eliminar juego → DELETE
10. Probar errores:
 - Games sin token → 401
 - Register email duplicado → 409
 - Login incorrecto → 401
 - Body inválido → 400 con errores Zod
 - ID inexistente → 404

11.3 Token en Bruno

Los juegos ahora requieren el header:

```
Authorization: Bearer {{token}}
```

Puedes usar una variable de entorno en Bruno:

1. En Bruno, ve a **Environments**
 2. Crea una variable `token`
 3. Pega el token que recibes de Register o Login
 4. Las requests de Games usarán `{{token}}` automáticamente
-

12. Verificación en MySQL Workbench

Conecta a Aven con estos datos:

- **Host:** backlog-games-crmprueba.l.aivencloud.com
- **Puerto:** 14560
- **Usuario:** avnadmin
- **Password:** (AVNS_RrLuUnUZRtS2WduWZoF)
- **Database:** defaultdb

Consultas útiles

```
-- Usuarios registrados
SELECT id, email, alias, createdAt FROM User;

-- Juegos con su usuario
SELECT g.id, g.name, g.category, g.priorityScore, g.completed, u.alias AS usuario
FROM Game g
JOIN User u ON u.id = g.userId;

-- Juegos de un usuario específico
SELECT id, name, category, priorityScore, completed
FROM Game
WHERE userId = 1;

-- Juegos completados con su valoración
SELECT g.name, g.completedAt, g.rating, g.notes, u.alias
FROM Game g
JOIN User u ON u.id = g.userId
WHERE g.completed = true;

-- Estructura de las tablas
DESCRIBE User;
DESCRIBE Game;

-- Ver la relación (clave foránea)
SELECT CONSTRAINT_NAME, TABLE_NAME, COLUMN_NAME, REFERENCED_TABLE_NAME,
REFERENCED_COLUMN_NAME
FROM INFORMATION_SCHEMA.KEY_COLUMN_USAGE
WHERE TABLE_SCHEMA = 'defaultdb' AND REFERENCED_TABLE_NAME IS NOT NULL;
```

13. Capturas recomendadas

Para tu PDF de entrega, estas son las capturas que te recomiendo:

13.1 Variables de entorno

- `.env` con `JWT_SECRET`, `JWT_EXPIRES_IN`, `DATABASE_URL`

13.2 Base de datos

- `DESCRIBE User;` en MySQL Workbench
- `DESCRIBE Game;` mostrando la columna `userId`
- Resultado de `SELECT` con `JOIN` entre `User` y `Game`

13.3 Código clave

- `src/errors/index.js` — Clases de error
- `src/utils/jwt.js` — Helpers JWT
- `src/middlewares/auth.middleware.js` — Protección de rutas
- `src/modules/auth/auth.useCases.js` — Use cases de auth
- `src/modules/auth/auth.service.js` — Service de auth (solo Prisma)
- `src/modules/games/games.useCases.js` — Use cases de games (ver ownership)
- `src/modules/games/games.service.js` — Service de games (solo Prisma)
- `src/middlewares/error.middleware.js` — Manejo de errores

13.4 Pruebas con Bruno (8-10 capturas)

- **Register** → 201 con token y user
- **Register duplicado** → 409
- **Login** → 200 con token
- **Login incorrecto** → 401
- **Games sin token** → 401
- **Crear juego con token** → 201 con `userId`
- **Listar juegos** → 200 solo juegos del usuario
- **Editar juego** → 200 con prioridad recalculada
- **Completar juego** → 200 con fecha y rating
- **404 en juego inexistente** → 404

13.5 Error de validación Zod

- POST a games con body vacío → 400 con errores por campo

Fin del Sprint 2 — API con autenticación JWT, multi-usuario y arquitectura profesional con use cases.